CIS*3110 Operating Systems

Week 1

Lecture #2 Intro to Operating Systems

Prof. Yulia Kotseruba

January 8, 2026

Quiz 0

- **64%** of students passed
- **3** students got **90%** correct
- The hardest questions were **Bash-related**

8%
- What is the correct way of defining a variable in Bash
  a) `$course=cis3110` b) `course=cis3110` c) `course = cis3110`

16%
- Which command will output 4 in Bash
  a) `echo 2+2` b) `echo ${2 + 2}` c) `echo $((2+2))`

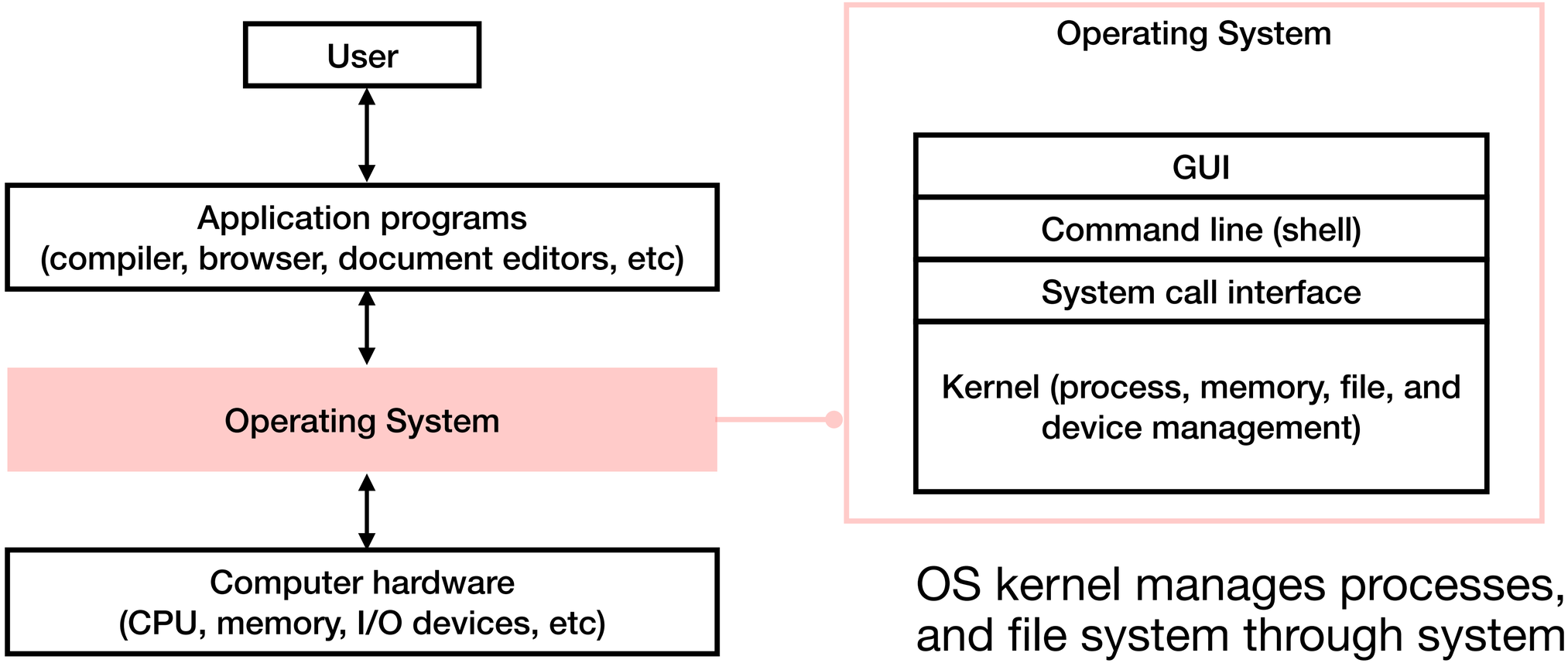
26 %
- Which command will take you to the `cis3110` folder located in your home directory a) `cd cis3110` b) `cd /cis3110` c) `cd ~/cis3110`

Recap: Course info

- **Contact:** cis3110**a**@**socs**.uoguelph.ca + Teams
- **Course structure:** 2x lectures + 1x lab per week
- **Assessments:** 4 assignments + 8 labs + midterm + final + 20 in-class quizzes
- **Grading:** 40% assignments + 10% labs + 20% midterm + 30% final + 5% bonus
- **Late submission:** 4 grace days for assignments (for the whole term)
- **Plagiarism:** do not cheat, do not use LLMs to do your homework
- **How to succeed:** do a little every day rather than cram before exams + get all "easy" grades (labs and quizzes) + attend classes (repetition and recall)

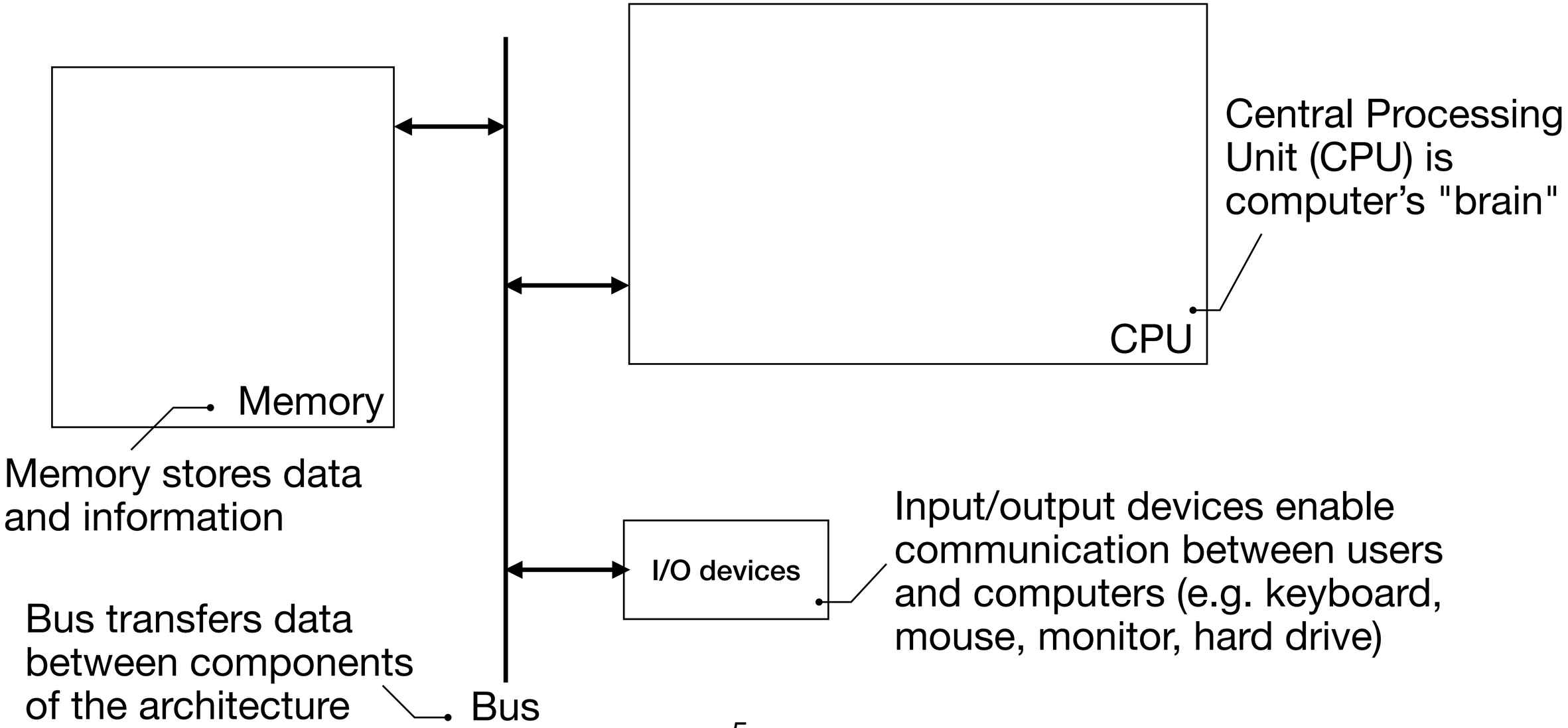
Recap: OS

- OS is an intermediary component between the users and the computer hardware

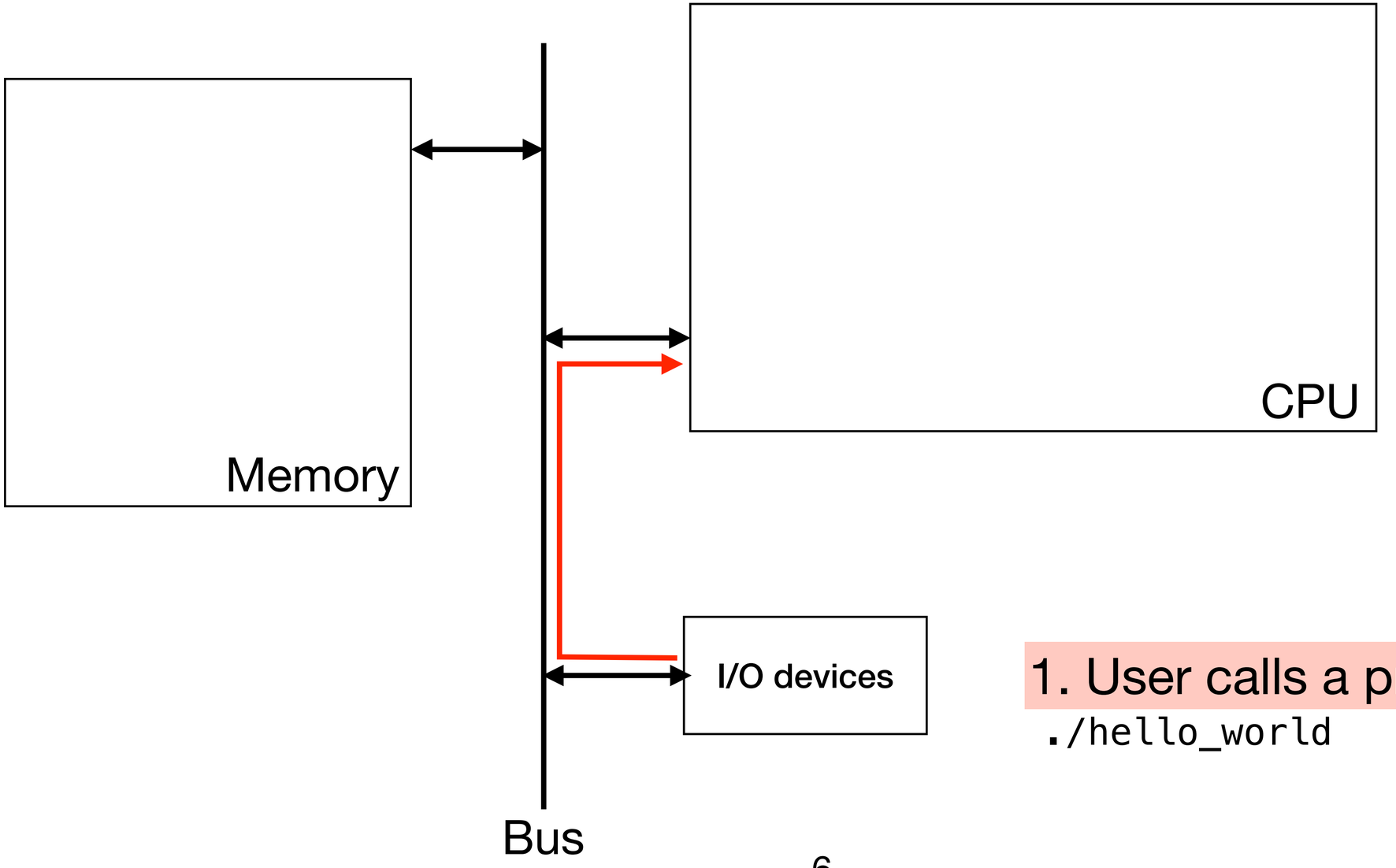


OS kernel manages processes, memory, and file system through system call API that interacts with computer hardware

A typical computer architecture



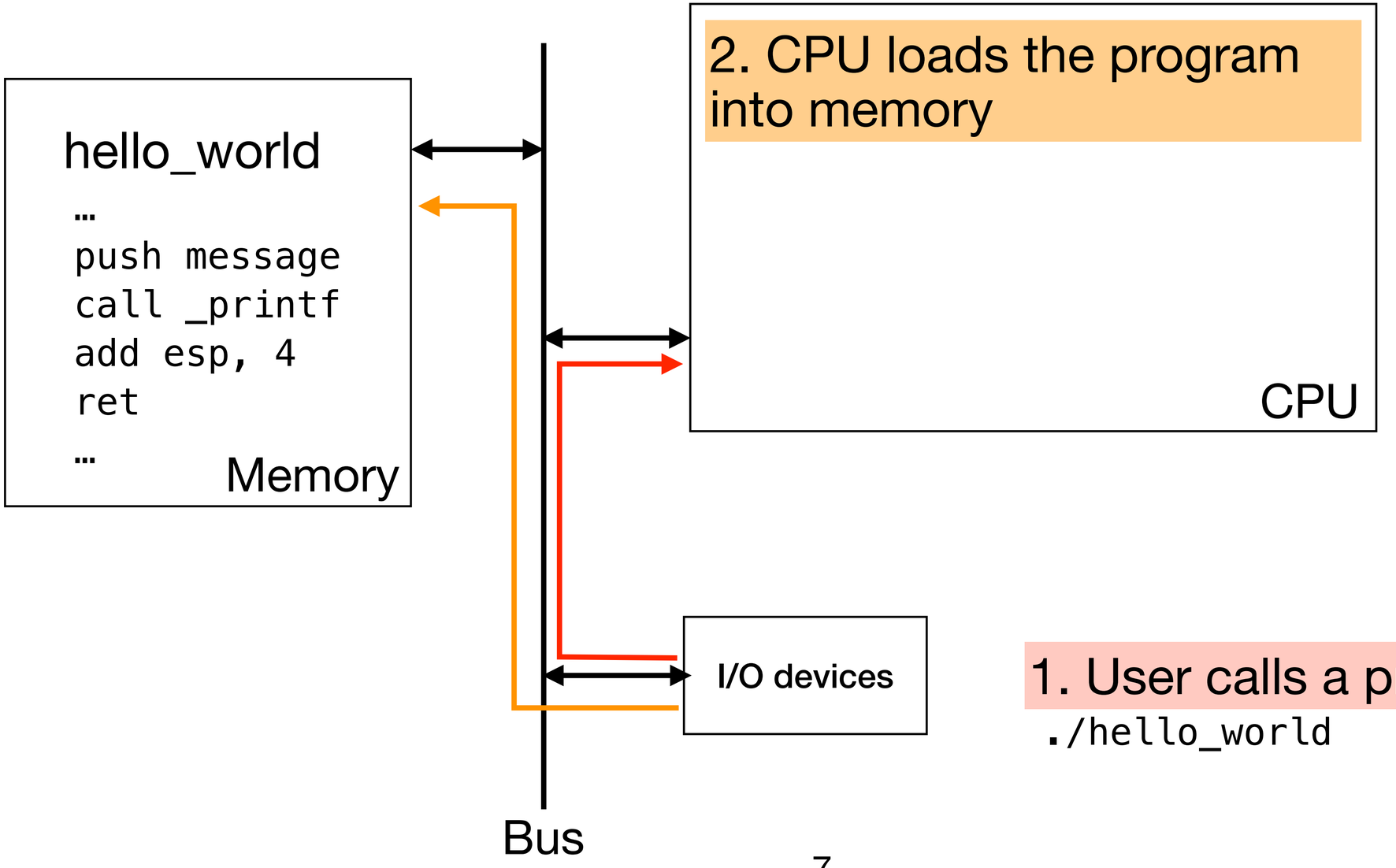
What happens when the program starts?



1. User calls a program

```
./hello_world
```

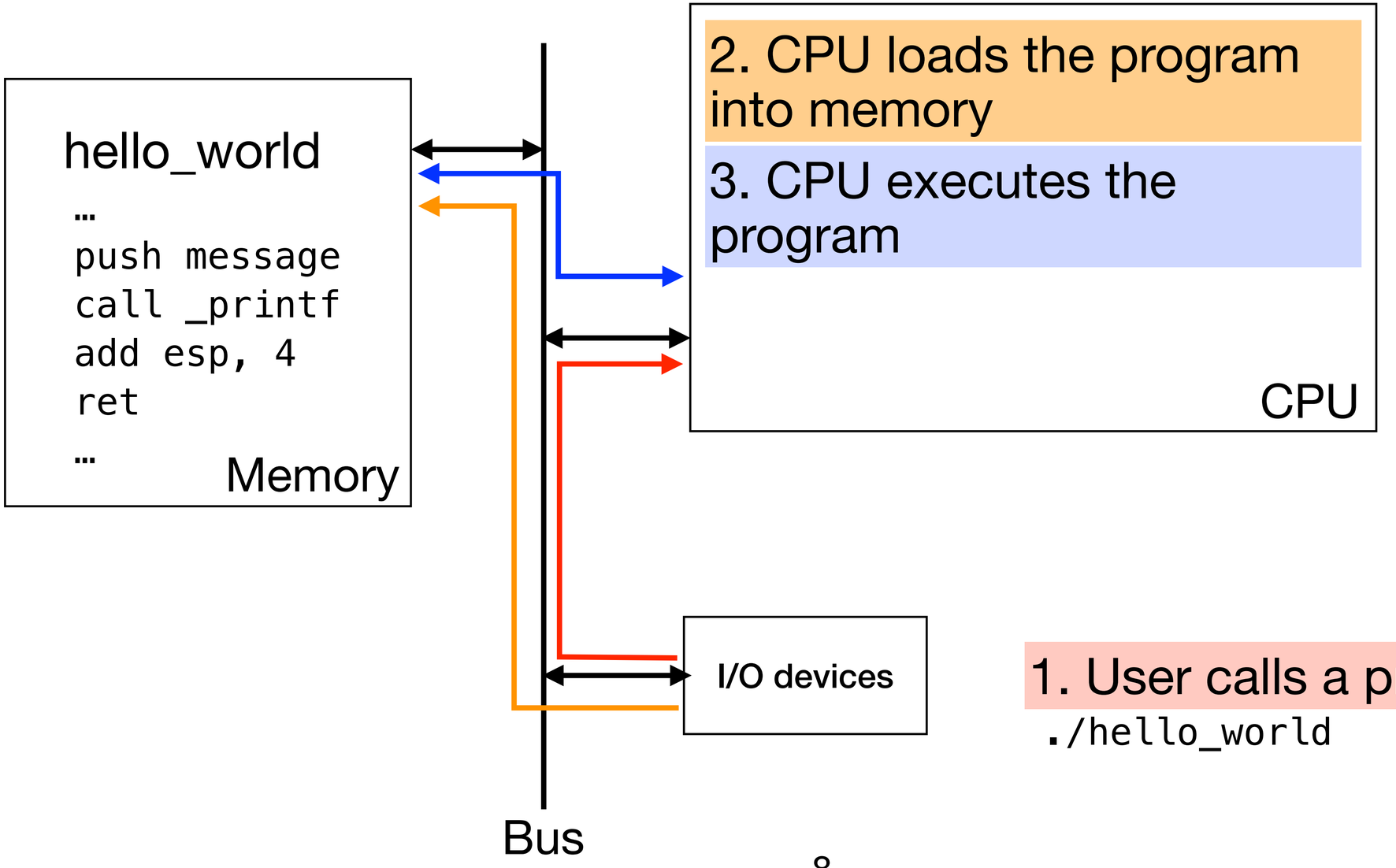
What happens when the program starts?



1. User calls a program

```
./hello_world
```

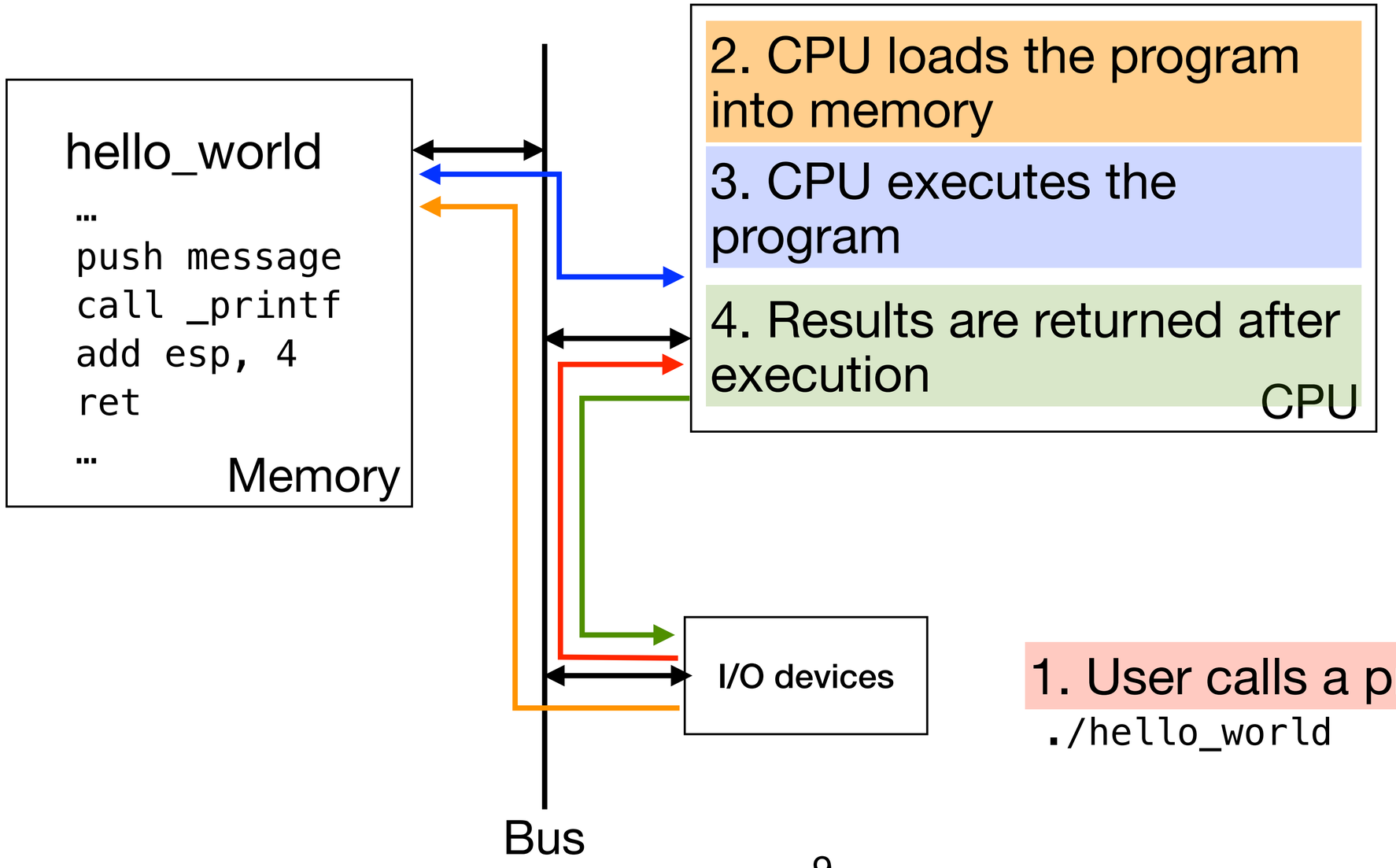
What happens when the program starts?



1. User calls a program

```
./hello_world
```

What happens when the program starts?



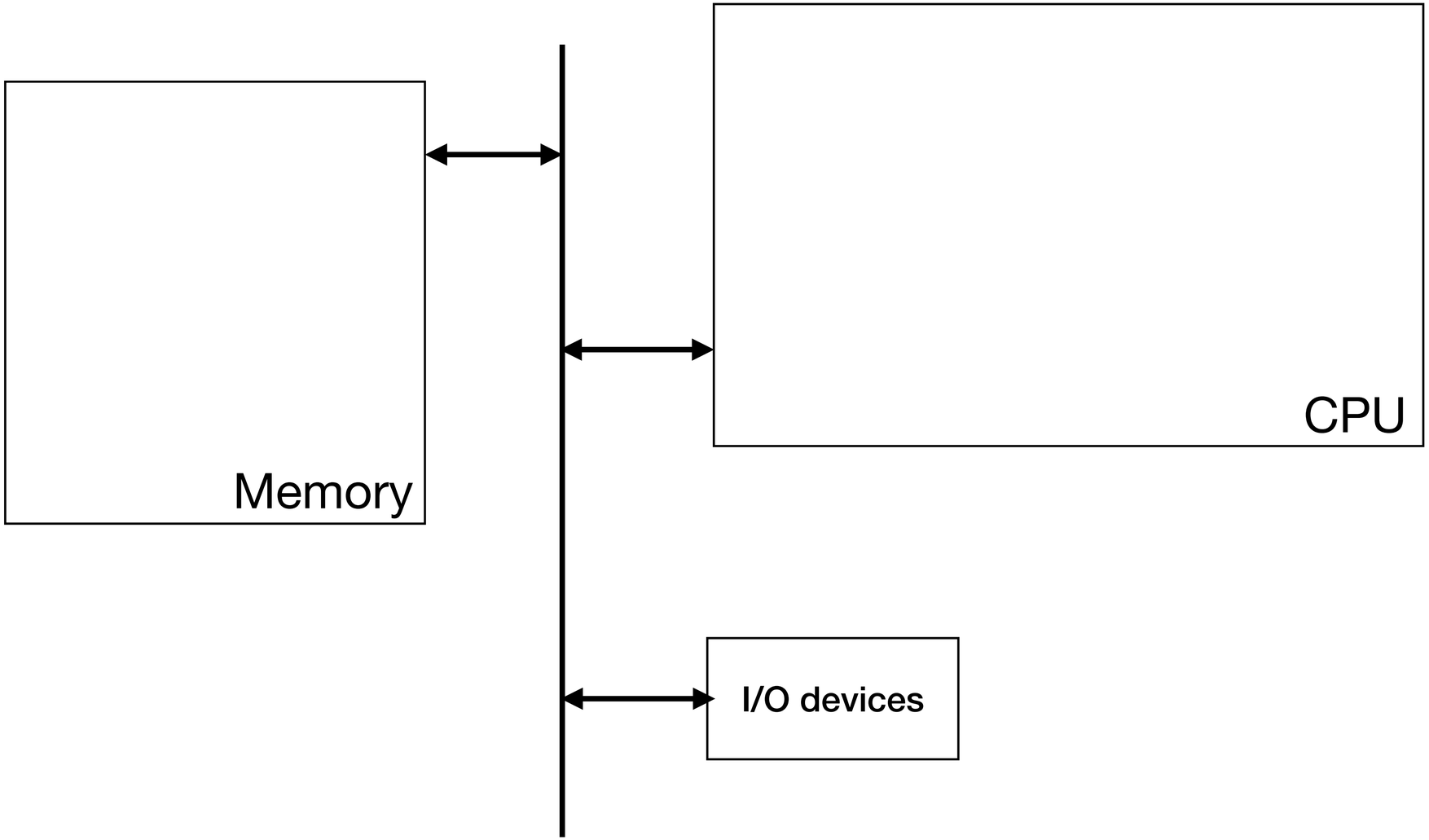
1. User calls a program

```
./hello_world
```

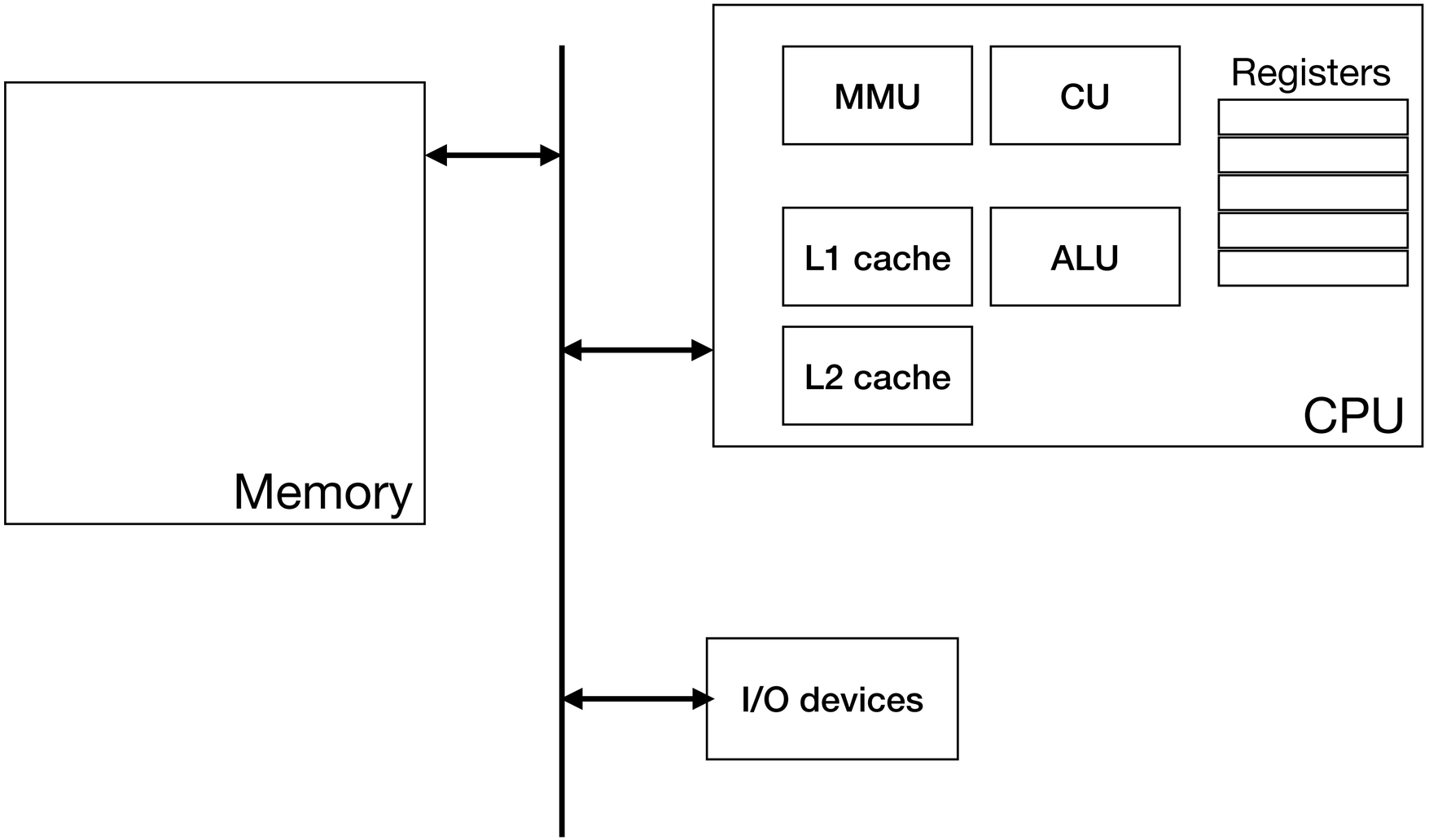
What happens when the program starts?

- When a user runs a program via a terminal or UI
  - The program is loaded into memory
  - Executed line by line in the CPU
  - Upon completion, the results are returned to user (or saved, discarded, etc)

A typical computer architecture

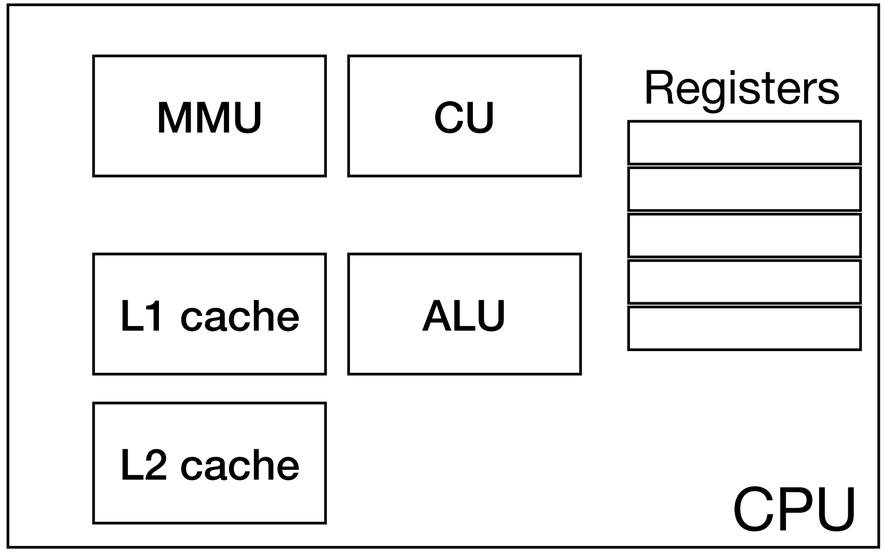


CPU



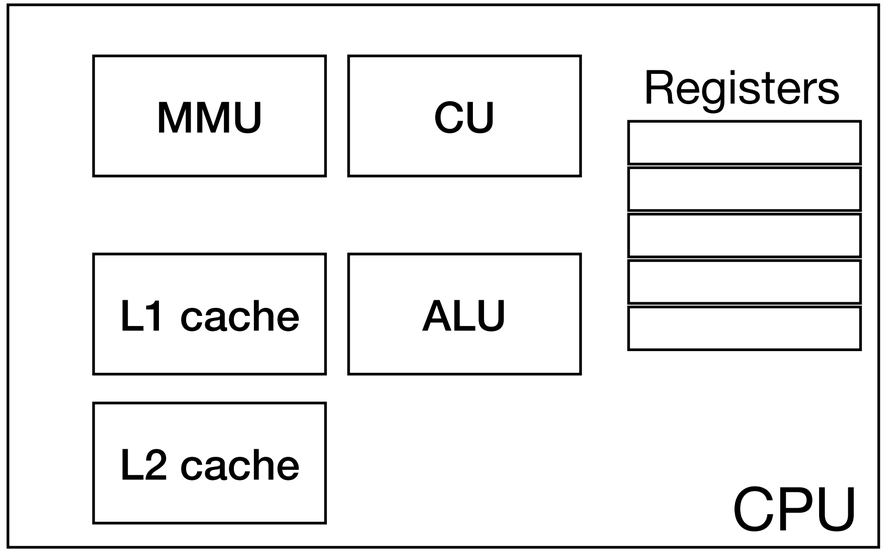
CPU

- **Control Unit (CU)** directs data flow and decodes instructions
- **Arithmetic Logic Unit (ALU)** executes math and logical operations
- **Memory Management Unit (MMU)** manages memory addressing
- **L1/L2 cache** stores frequently used data for faster processing
- **Registers** hold data needed for executing the program

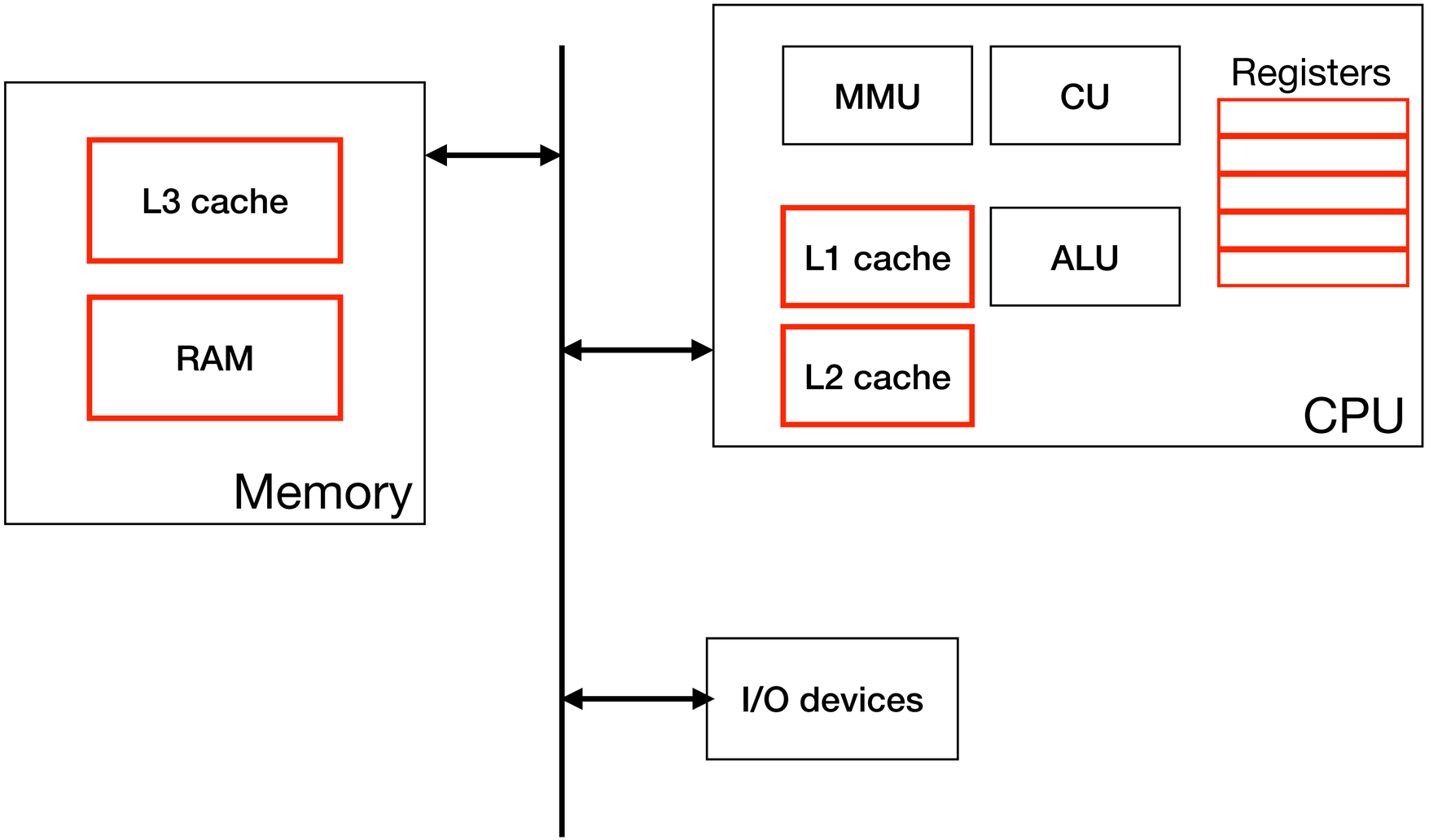


CPU Registers

- **User-visible** - can be directly manipulated by the program/programmer
- **Control and status registers** - usually inaccessible to the users, reserved for OS operation
  - **Program Counter (PC)** - holds the address of the next instruction
  - **Instruction Register (IR)** - contains the current instruction being executed
  - **Program Status Word (PSW)** - register(s) holding status information (flags, interrupts)
  - **Memory address registers**

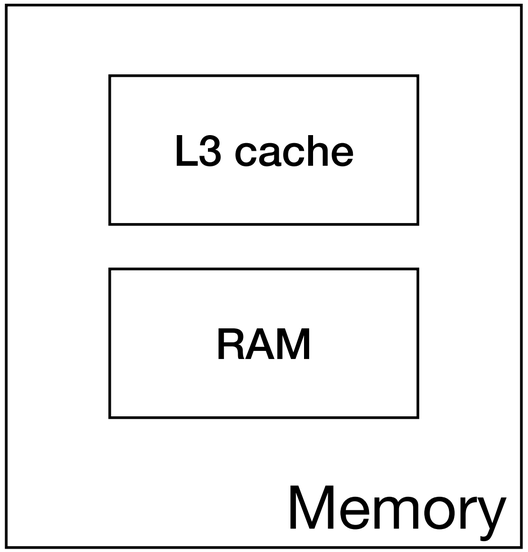


Memory


L3 cache
RAM
Memory
MMU
CU
Registers
L1 cache
ALU
L2 cache
CPU
I/O devices

Memory

- **L1/L2 cache and registers** - for the most frequently used data
- **L3 cache** - cache for storing less frequently accessed data used by CPU(s)
- **Random-access memory (RAM)** - high-speed volatile (temporary) storage for all running tasks and processes

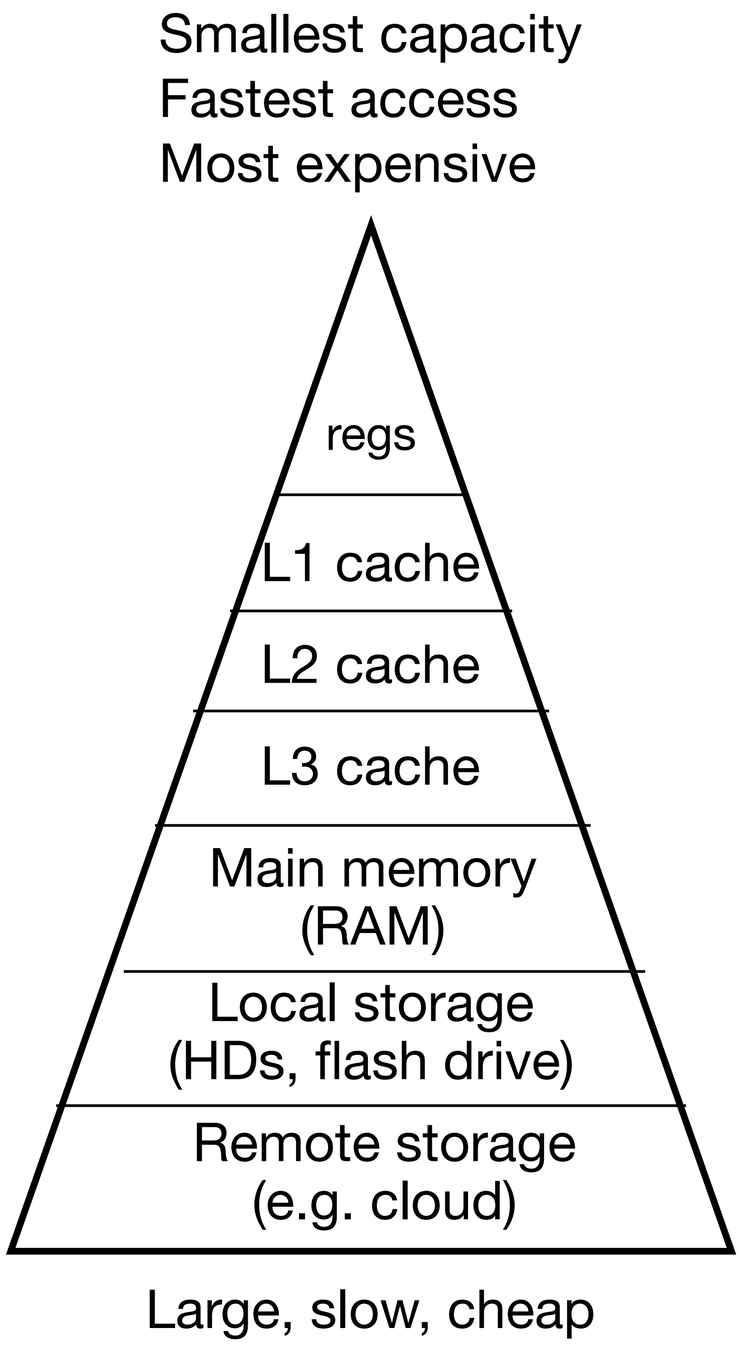


random-access - data can be stored and retrieved from anywhere without speed penalty

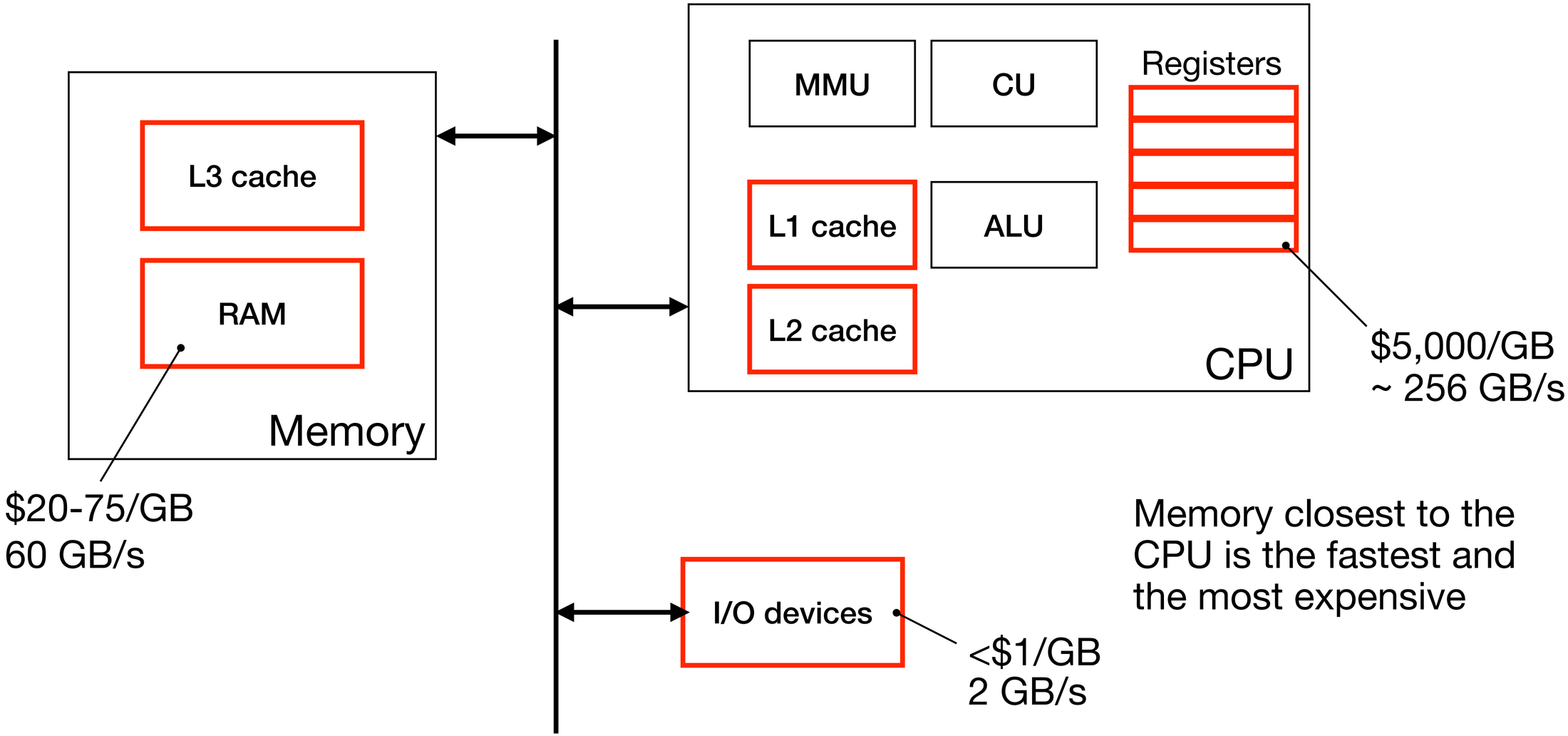
volatile - retains information only when powered

Memory hierarchy

- Most modern CPUs are so fast that **memory is a bottleneck** for most computations
- **Memory hierarchy** helps improve performance by minimizing access time
- Each level holds information retrieved from the level above
- OS manages data storage and retrieval across all levels of memory hierarchy

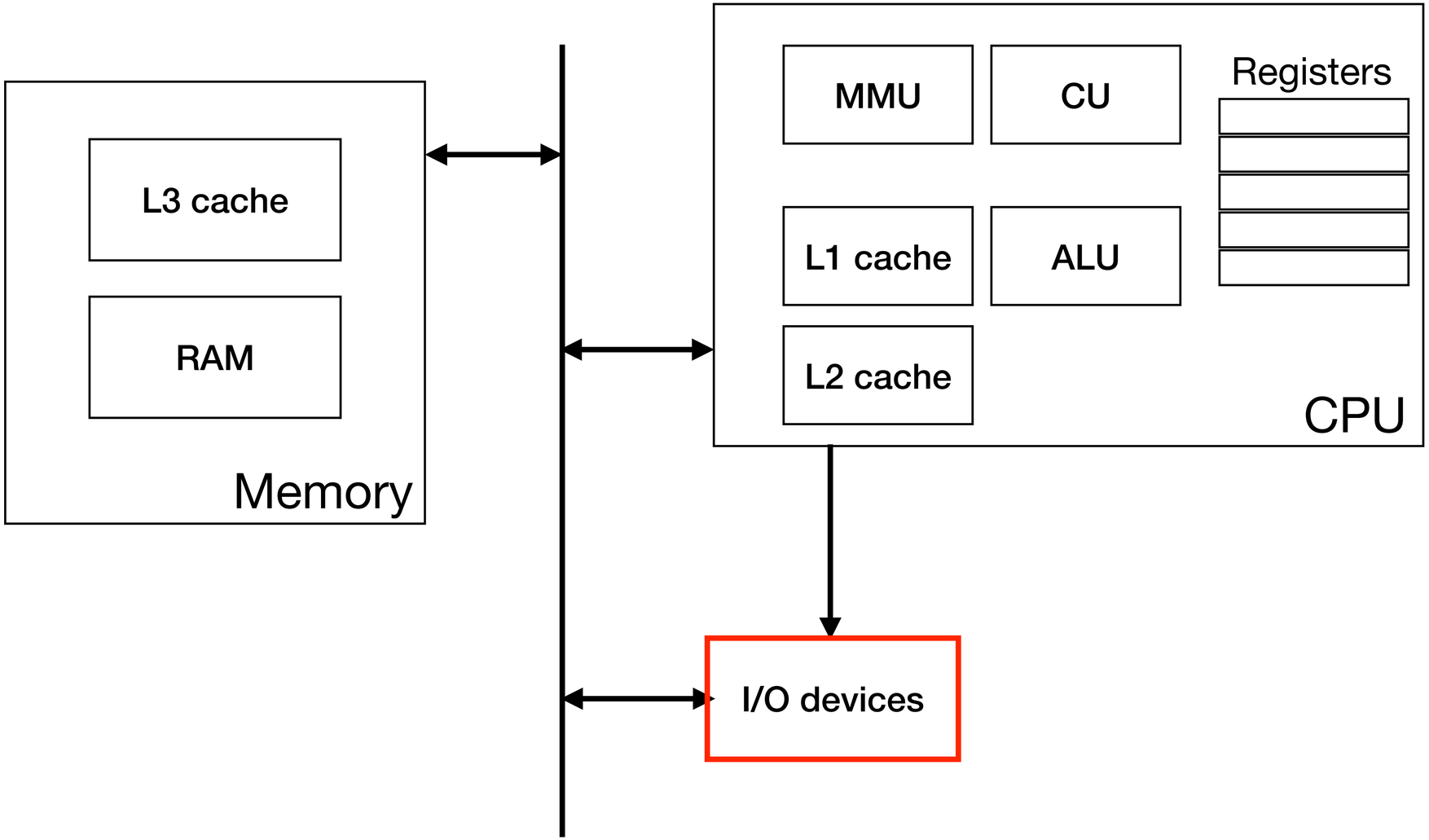


Memory hierarchy

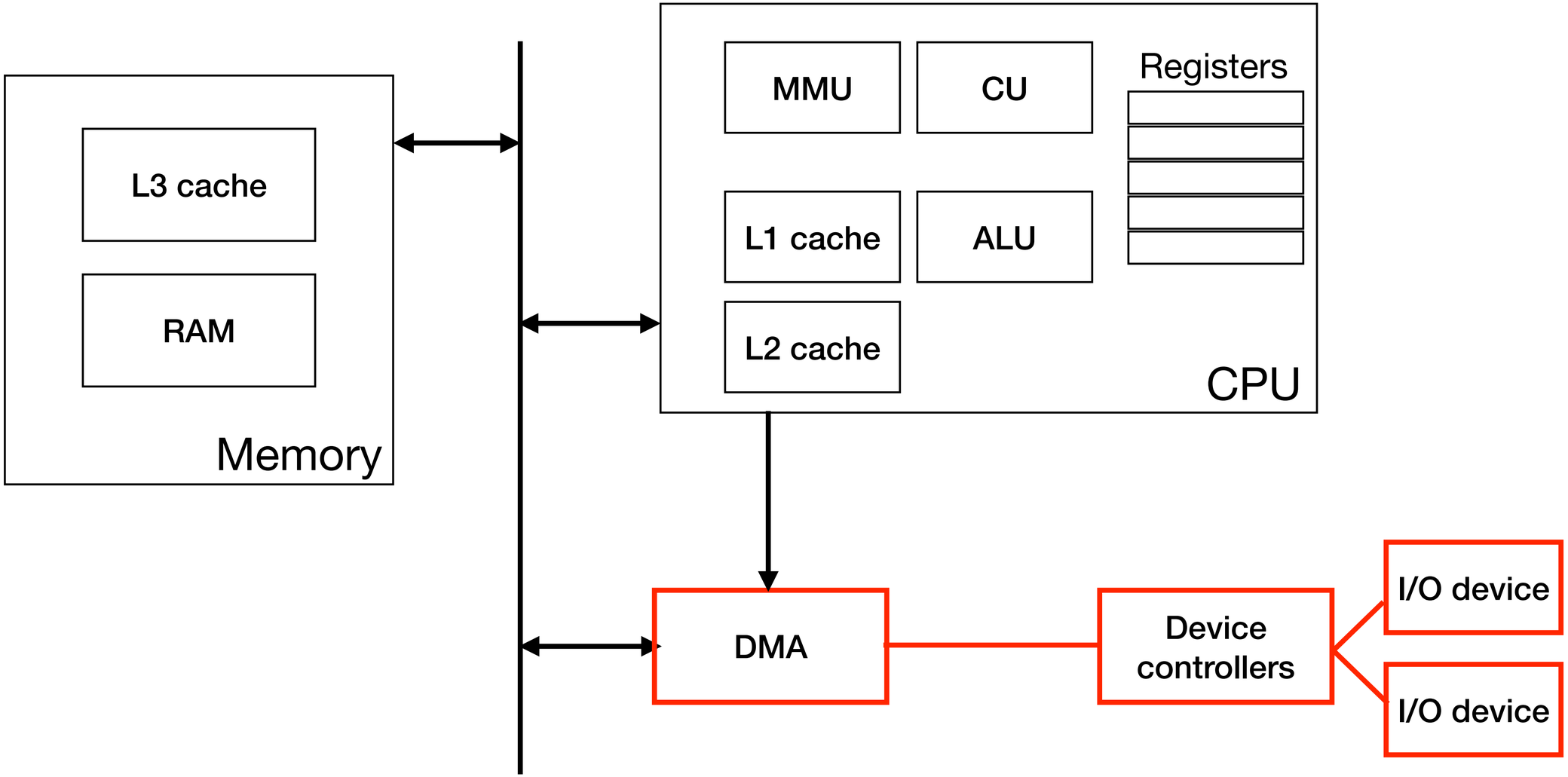


Memory closest to the CPU is the fastest and the most expensive

I/O devices

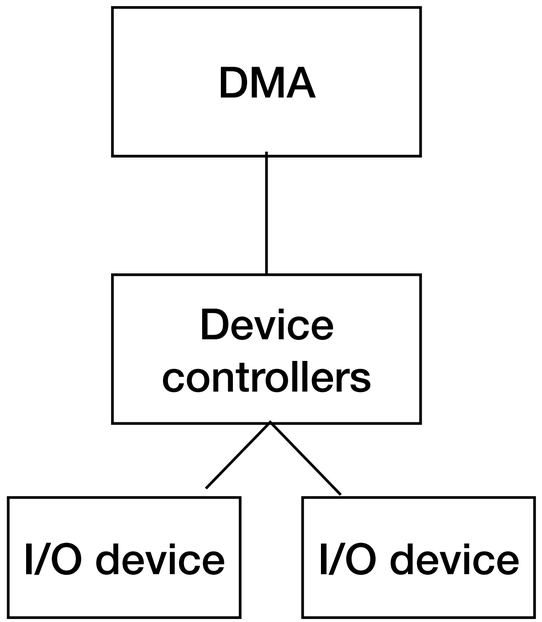

L3 cache
RAM
Memory
MMU
CU
Registers
L1 cache
ALU
L2 cache
CPU
I/O devices

I/O devices

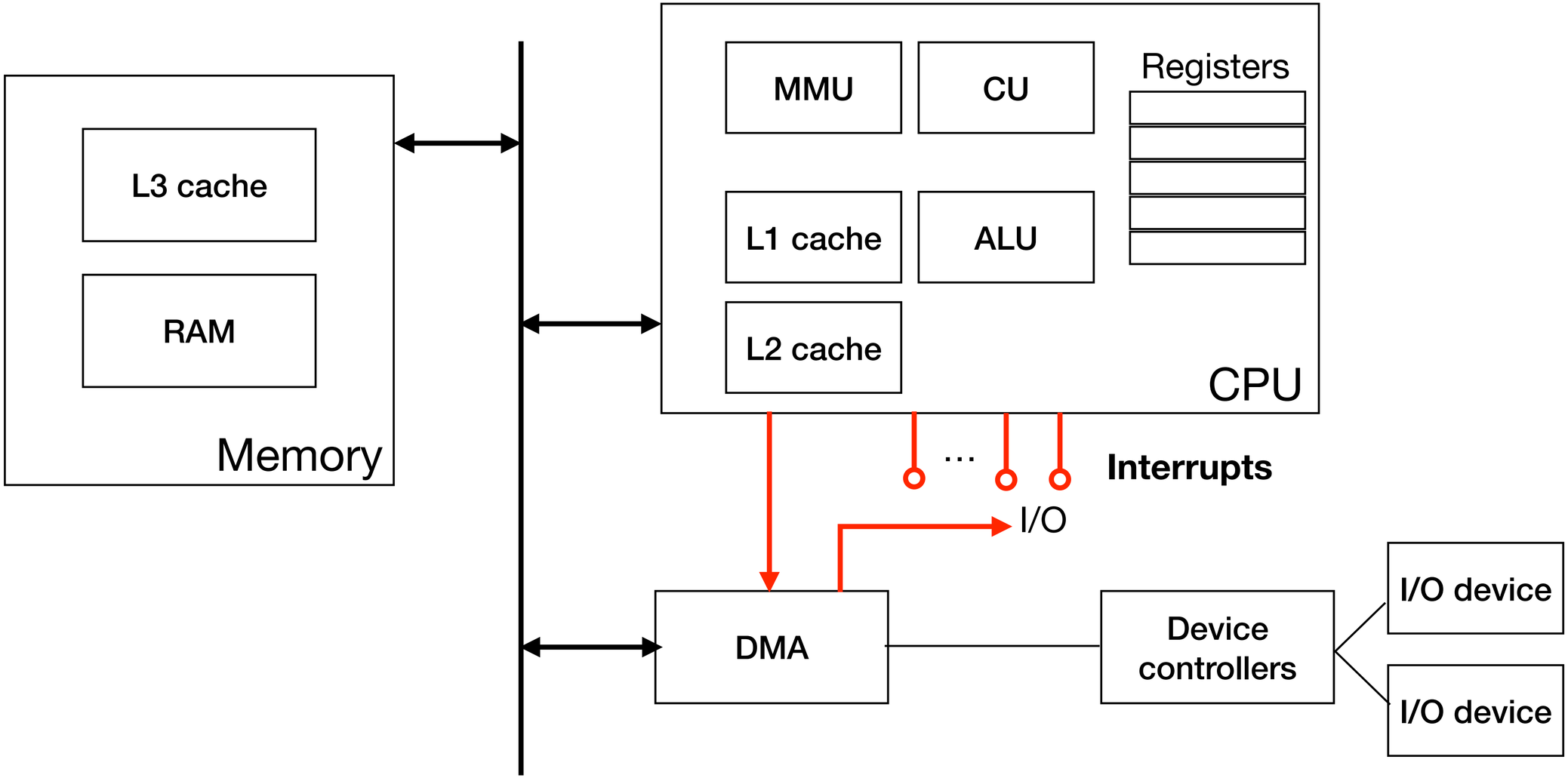

L3 cache
RAM
Memory
MMU
CU
Registers
L1 cache
ALU
L2 cache
CPU
DMA
Device controllers
I/O device
I/O device

I/O devices

- **Direct Memory Access (DMA)** - orchestrates transfers between devices and main memory without CPU intervention
- **Device controllers** - an interface between the OS and I/O devices
- **I/O devices** - keyboards, monitors, storage, etc.



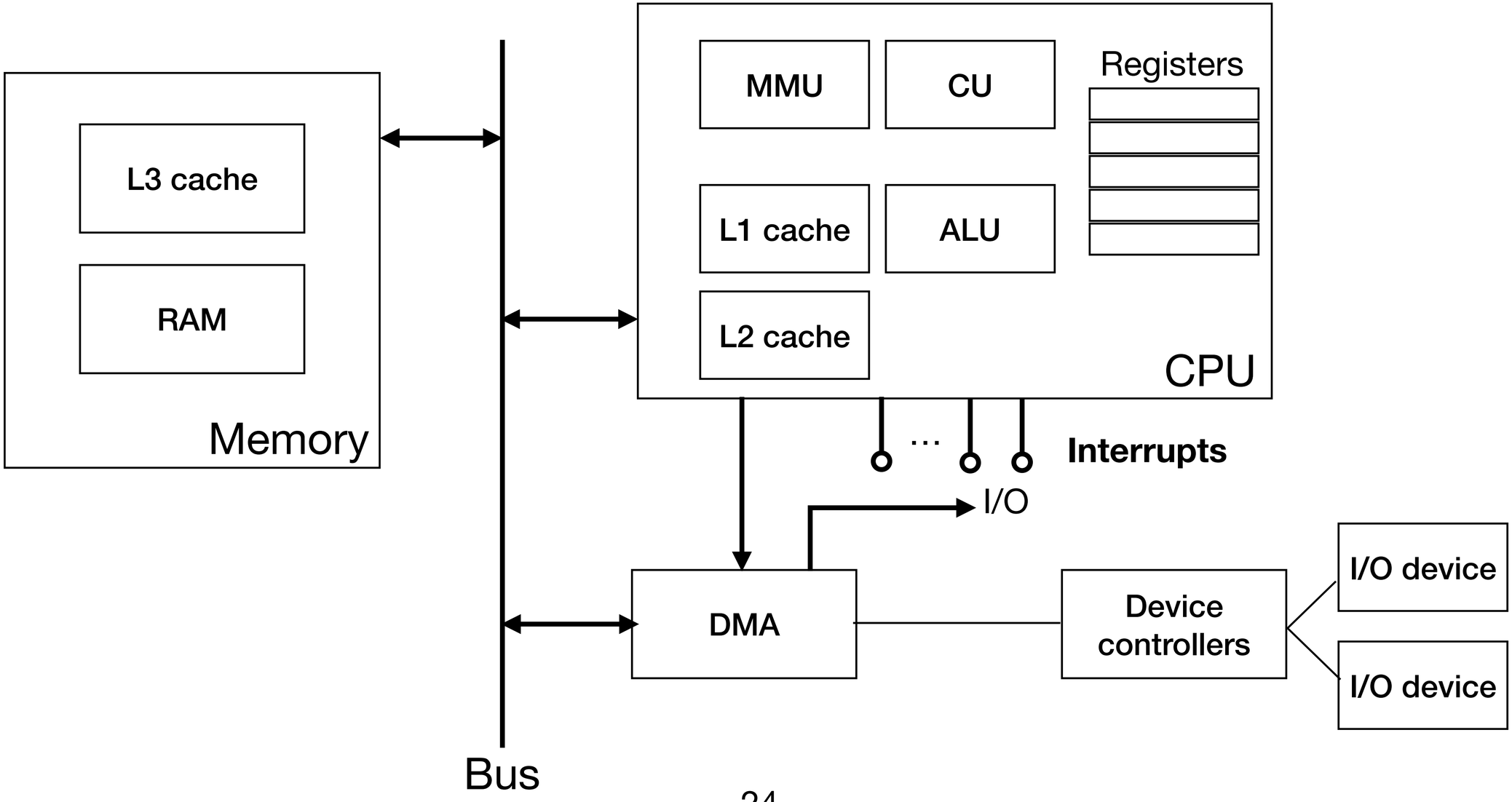
Interrupts


L3 cache
RAM
Memory
MMU
CU
Registers
L1 cache
ALU
L2 cache
CPU
...
Interrupts
I/O
DMA
Device controllers
I/O device
I/O device

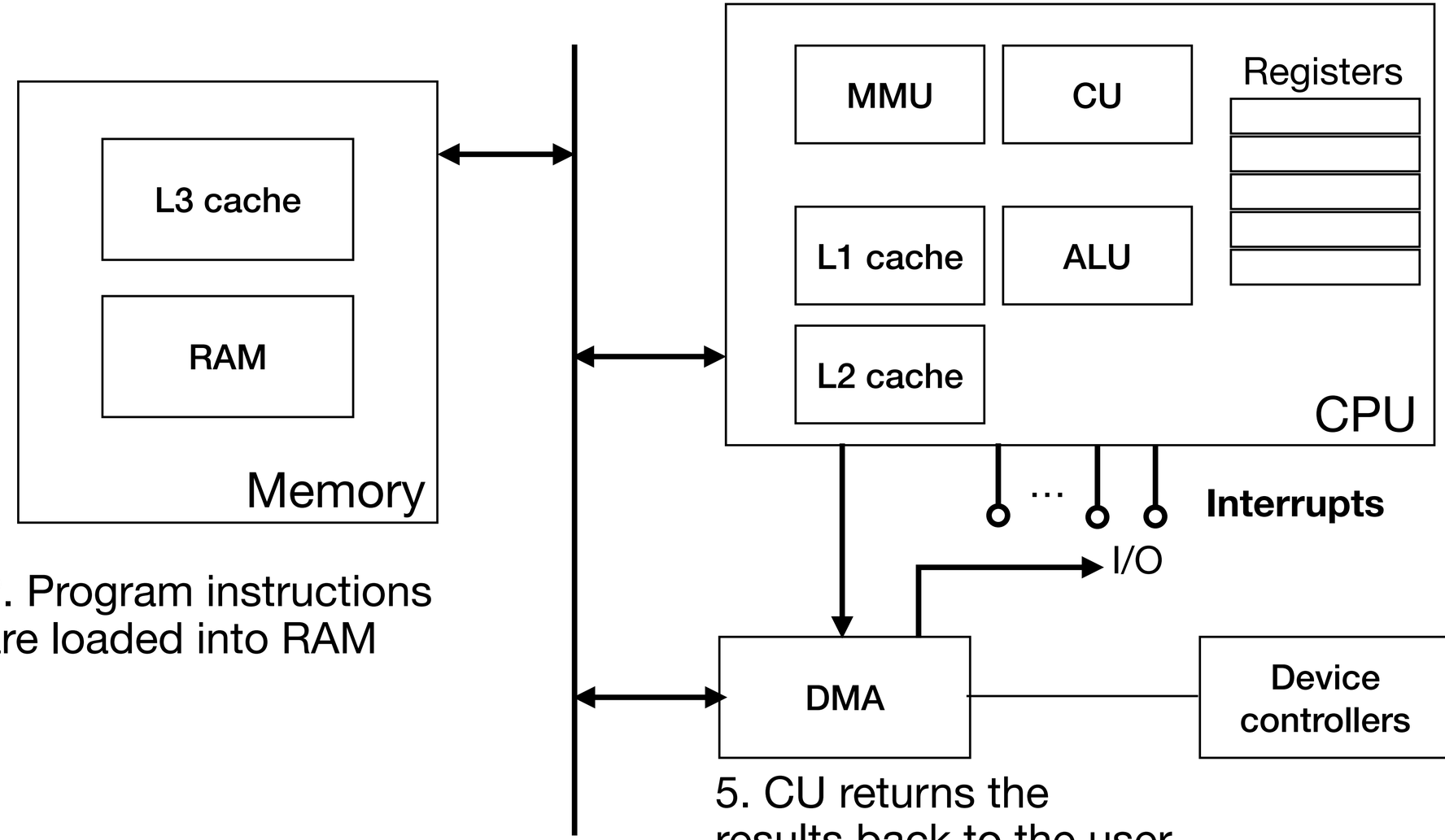
Interrupts

- Interrupts are signals issued in response to hardware or software events
- **Software interrupts** - synchronous, generated by the currently running processes
  - e.g. division by zero, system call
- **Hardware interrupts** - asynchronous, caused by external device, can arrive at any time, can be masked (ignored)
  - e.g. pressing a keyboard key, moving a mouse

A typical computer architecture (detailed)



What happens when the program starts?



3. CU fetches instructions one by one, decodes them, populates the registers, and sends signals to ALU, MMU, I/O

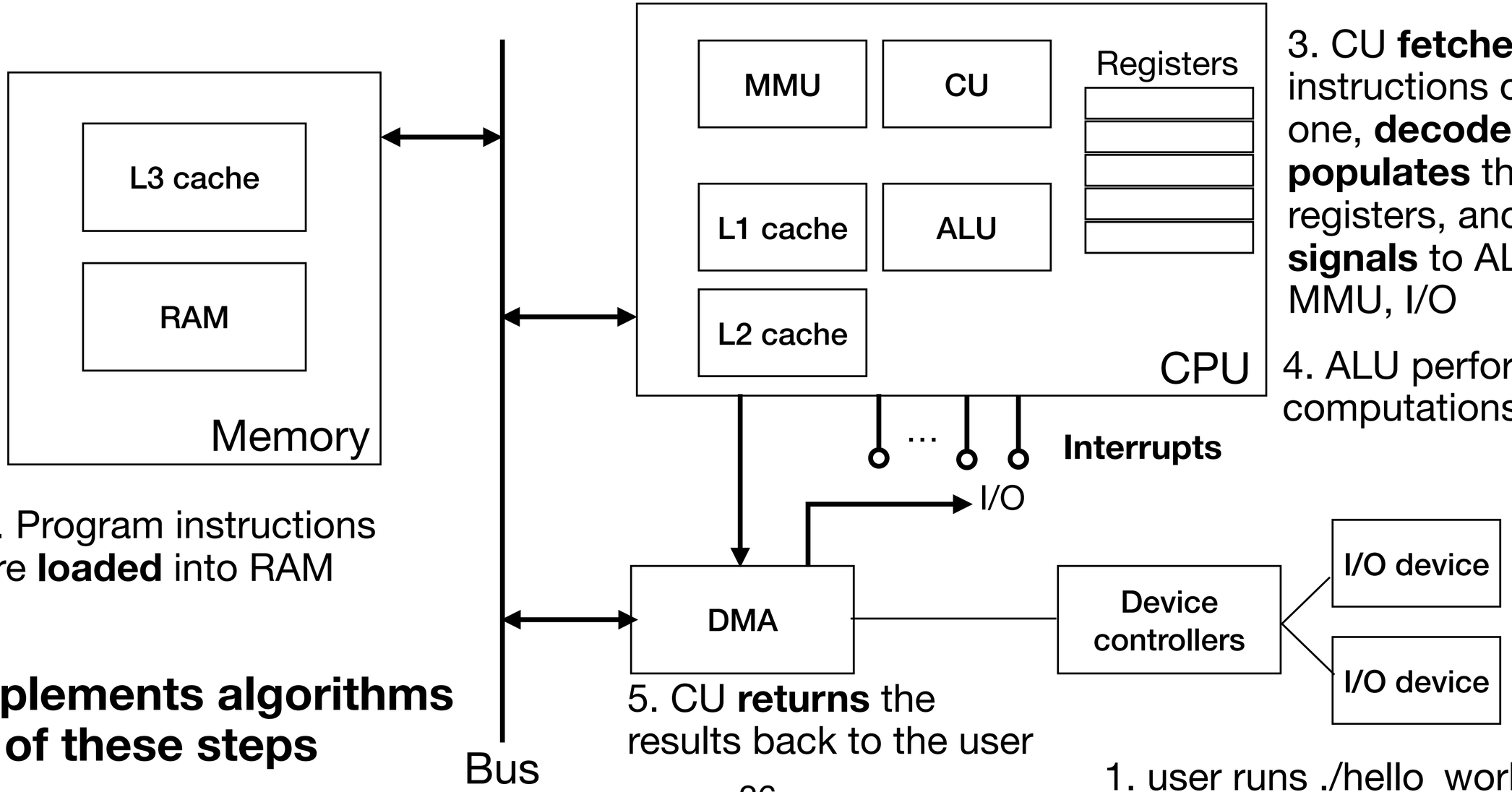
4. ALU performs computations

2. Program instructions are loaded into RAM

5. CU returns the results back to the user

1. user runs ./hello_world

What happens when the program starts?



3. CU **fetches** instructions one by one, **decodes** them, **populates** the registers, and **sends signals** to ALU, MMU, I/O

4. ALU performs computations

2. Program instructions are **loaded** into RAM

5. CU **returns** the results back to the user

1. user runs ./hello_world

OS implements algorithms for all of these steps

Role of an OS

- When a user runs a program via terminal or UI **(OS manages I/O)**
  - Program is loaded into memory **(OS loader)**
  - Executed line by line in the CPU **(OS fetch-decode-execute cycle)**
  - Upon completion, the results are returned to user/saved/discarded **(OS reclaims resources and manages I/O if needed)**

How does an OS operate?

- Operating system exists to make running programs on computer hardware more efficient and more convenient for the user
- Three general techniques are used to achieve this
  - **virtualization**
  - **concurrency**
  - **persistence**

Virtualization

- Turns a physical resource into a virtual form of itself
- Allows effective resource management by letting many programs
  - to run concurrently (share the CPU)
  - to access the same instructions/data (share the memory)

Virtualization: CPU

cpu.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
  if (argc != 2) {
    fprintf(stderr, "usage: cpu<string>\n");
    exit(1);
  }
  char *str = argv[1];
  while (1) {
    Spin(1);
    printf("%s\n", str);
  }
  return 0;
}
```

What does this program do?

In an infinite loop:

Spin(1) continuously checks the time and returns after 1 second elapsed

printf outputs the first argument of the function

Let's check

```
>>> gcc -o cpu cpu.c -Wall
>>> ./cpu "A"
A
A
A
A
^C
>>>
```

break out of the infinite loop with Ctrl+C

Virtualization: CPU

cpu.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
  if (argc != 2) {
    fprintf(stderr, "usage: cpu<string>\n");
    exit(1);
  }
  char *str = argv[1];
  while (1) {
    Spin(1);
    printf("%s\n", str);
  }
  return 0;
}
```

What does appending an "&" at the end of the command do in terminal?

```
>>> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
```

Executes command in the background and immediately returns to the command line

Allows running multiple process concurrently from a single shell session

Virtualization: CPU

cpu.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
  if (argc != 2) {
    fprintf(stderr, "usage: cpu<string>\n");
    exit(1);
  }
  char *str = argv[1];
  while (1) {
    Spin(1);
    printf("%s\n", str);
  }
  return 0;
}
```

```
>>> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
C
A
B
D
^C
>>>
```

Output shows 4 different process ids

This is what CPU virtualization is — turning one physical CPU into many virtual ones

OS does this by constantly switching between these different programs

Virtualization: memory

mem.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
    int *p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) address pointed by p: %p\n",
            getpid(), p);
    p* = (int) getpid();
    while (1) {
      Spin(1);
      *p = *p + 1;
      printf("(%d) p: %d\n", getpid(), *p);
    }
    return 0;
}
```

What does this program do?

Allocates memory and prints its address and pid of the process

stores the pid

In an infinite loop:

sleeps for 1 s

increments the stored value

prints the PID and the value

Virtualization: memory

mem.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
    int *p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) address pointed by p: %p\n",
            getpid(), p);
    p* = (int) getpid();
    while (1) {
      Spin(1);
      *p = *p + 1;
      printf("(%d) p: %d\n", getpid(), *p);
    }
    return 0;
}
```

```
>>> ./mem
(2134) address pointed to by p: 0x57fe31f672a0
(2134) p: 2135
(2134) p: 2136
(2134) p: 2137
^C
>>>
```

This is what the output looks like

Virtualization: memory

mem.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
    int *p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) address pointed by p: %p\n",
            getpid(), p);
    p* = (int) getpid();
    while (1) {
      Spin(1);
      *p = *p + 1;
      printf("(%d) p: %d\n", getpid(), *p);
    }
    return 0;
}
```

What happens if we run two copies of this program concurrently?

PIDs are different — these are two independent processes

```
>>> ./mem & ./mem &
(2718) address pointed by p: 0x5555555592a0
(2719) address pointed by p: 0x5555555592a0
(2718) p: 2719
(2719) p: 2720
(2718) p: 2720
(2719) p: 2721
(2718) p: 2721
(2719) p: 2722
(2718) p: 2722
(2719) p: 2723
^C
```

2719 and 2720 appear to be stored at the same address

Virtualization: memory

```
#include <stdio.h>                                  cpu.c
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
    int *p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) address pointed by p: %p\n",
            getpid(), p);
    p* = (int) getpid();
    while (1) {
      Spin(1);
      *p = *p + 1;
      printf("(%d) p: %d\n", getpid(), *p);
    }
    return 0;
}
```

What happens if we run two copies of this program concurrently?

PIDs are different — these are two independent processes

```
>>> ./mem & ./mem &
(2718) address pointed by p: 0x5555555592a0
(2719) address pointed by p: 0x5555555592a0
(2718) p: 2719
(2719) p: 2720
(2718) p: 2720
(2719) p: 2721
(2718) p: 2721
(2719) p: 2722
(2718) p: 2722
(2719) p: 2723
^C
```

2719 and 2720 appear to be stored at the same address

This is memory virtualization — each program has its own (virtual) address space, which is mapped by the OS to two different physical locations

For this to work, turn-off address-space layout randomization (ASLR)

Virtualization summary

- Turns a **physical resource** into a **virtual** form of itself - illusion that there is potentially infinite resources available
- Allows **effective resource management** by letting many programs
  - to run concurrently (share the CPU — illusion of many CPUs)
  - to access instructions/data (share the memory — illusion of infinite memory)

Concurrency

```
// includes omitted for space
volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
    counter++; }

int main(int argc, char *argv[]) {
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf('Initial value: %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value: %d\n", counter);
    return 0
}
```

What does this program do?

It creates 2 threads using Pthread_create()

each thread starts a routine worker() which increments a counter for loops number of times

```
>>> ./threads 1000
Initial value: 0
Final value: 2000
```

if we run this, we expect two workers to each increment the counter for N=1000, so the output should be 2000

Concurrency

```
// includes omitted for space
volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
    counter++; }

int main(int argc, char *argv[]) {
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf('Initial value: %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value: %d\n", counter);
    return 0
}
```

What if we run this with 100,000 as input?

```
>>> ./threads 100000
Initial value: 0
Final value: 143012

>>> ./threads 100000
Initial value: 0
Final value: 137298
```

Concurrency

```
// includes omitted for space
volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
    counter++; }

int main(int argc, char *argv[]) {
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf('Initial value: %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value: %d\n", counter);
    return 0
}
```

What if we run this with 100,000 as input?

```
>>> ./threads 100000
Initial value: 0
Final value: 143012

>>> ./threads 100000
Initial value: 0
Final value: 137298
```

Why is this happening? The values are wrong and different every time

Concurrency

```
// includes omitted for space
volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
    counter++; }

int main(int argc, char *argv[]) {
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf('Initial value: %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value: %d\n", counter);
    return 0
}
```

What if we run this with 100,000 as input?

```
>>> ./threads 100000
Initial value: 0
Final value: 143012

>>> ./threads 100000
Initial value: 0
Final value: 137298
```

Incrementing a value takes 3 instructions — loading from memory to register, adding 1 to it, and putting it back into memory — i.e. it is not **atomic**

2 workers do not coordinate and may access wrong values in memory as a result

Concurrency summary

- Situation when there are **multiple threads** within the **same memory space**
- Various problems may arise due to concurrency:
  - improper access to global resources
  - sub-optimal allocation of resources
  - processes interrupting, blocking, or interfering with each other
- OS helps solve these (and other) issues and **ensure correct execution** of the programs

Persistence

- On-board memory (RAM, registers, caches) is **volatile** - stores data only when powered and loses it once power is shut off
- Input/output (I/O) devices such as hard drives (HDs) or solid-state drives (SDDs) store data **persistently** (including the OS itself)
- **File system** is needed to organize and manage information on these devices
  - Translates name+offset to address on the hard drive
  - Keeps track of free space
  - Lots of file systems exist, this is still an active research area

Persistence

```
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <fcntl.h>
#include <sys/types.h>

int main(int argc, char *argv[]) {
    int fd = open("/tmp/file", O_WRONLY|
    O_CREAT|O_TRUNC, S_IRWXU);

    assert(fd > -1);

    int rc = write(fd, "hello world\n", 13);
    assert(rc == 13);
    close(fd);
    return 0;
}
```

What does this program do?

Creates a temp file and writes a "hello world" string into it

It makes 3 calls:

open() - opens the file or creates it

write() - writes data to the file

close() - closes the file

Persistence

- Writing to the file involves several operations from the OS
  - **open()** checks the **validity** of the path and **consistency** of flags with the file information (e.g. if the file is read-only, writing will not be allowed)
  - **open()** then either **locates** a file or **creates** a new one
  - **write()** will attempt to **write** the requested information and **allocate** more space on the disk if needed
  - **close()** will create a **new entry** for the file in its directory or **update** an existing **entry**
- All of these calls in turn will issue multiple I/O requests to the storage device
- The exact implementation depends on the type of file system and storage

Persistence summary

- **Persistence** is storing information **permanently** on a device
- User and system data are usually organized into a **file system** consisting of files and folders
- OS manages the file system and I/O devices used for storage

Summary

- OS is a middleware between user programs and computer hardware
  - Goal #1 convenience and needs of users running programs
  - Goal #2 use the hardware in the most efficient way possible
  - Goal #3 ensure correctness of the programs and prevent interference
- Three general techniques that most OSs use to achieve these goals are:
  - **Virtualization** of CPU and memory — sharing compute and memory across many processes
  - **Concurrency** — ensuring correctness during parallel execution of processes that share resources
  - **Persistence** — managing permanently stored information

Reading

- Chapter 2: Introduction to Operating Systems
  - https://pages.cs.wisc.edu/~remzi/OSTEP/intro.pdf
- Code from today's lecture
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/intro

Quiz 1

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet